

Why the training size N *regularizes model complexity*

Decoding one sentence from Section 4 [Nagler '23]

Gurmeher Singh Puri

Explainer deck — frequentist view of PFNs

The sentence we are decoding

From the PFN *approximation* section ([Sec. 4]):

“The training set sizes N_j can thus be seen
as a **regularizer on model complexity**.”

The sentence we are decoding

From the PFN *approximation* section ([Sec. 4]):

“The training set sizes N_j can thus be seen as a **regularizer on model complexity**.”

Three words to unpack:

N

the *size* of each simulated training
set we show the PFN during
pre-training

The sentence we are decoding

From the PFN *approximation* section ([Sec. 4]):

“The training set sizes N_j can thus be seen as a **regularizer on model complexity**.”

Three words to unpack:

N

the *size* of each simulated training
set we show the PFN during
pre-training

regularizer

a knob that limits how *complex* a
model is allowed to be (like ridge's
 λ , tree depth, bandwidth)

The sentence we are decoding

From the PFN *approximation* section ([Sec. 4]):

“The training set sizes N_j can thus be seen as a **regularizer on model complexity**.”

Three words to unpack:

N

the *size* of each simulated training
set we show the PFN during
pre-training

regularizer

a knob that limits how *complex* a
model is allowed to be (like ridge's
 λ , tree depth, bandwidth)

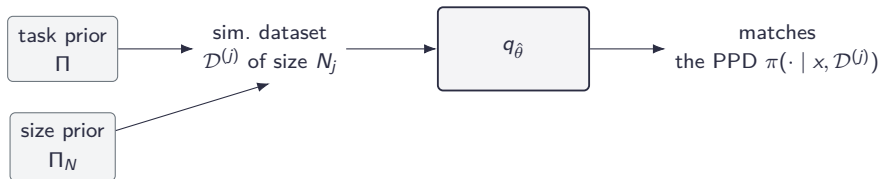
complexity

how *wiggly* a function the network
commits to producing

Claim: the *sizes* you train on secretly set the *complexity* of the predictions. Here is why.

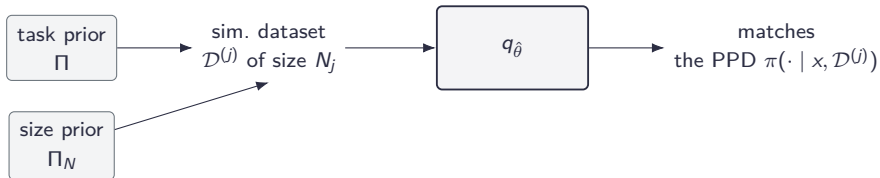
Setup: a PFN is trained to copy the PPD, at random sizes

During pre-training we repeatedly: **draw a task, simulate a dataset of random size N_j** , and nudge the net $q_{\hat{\theta}}$ to match that dataset's PPD.



Setup: a PFN is trained to copy the PPD, at random sizes

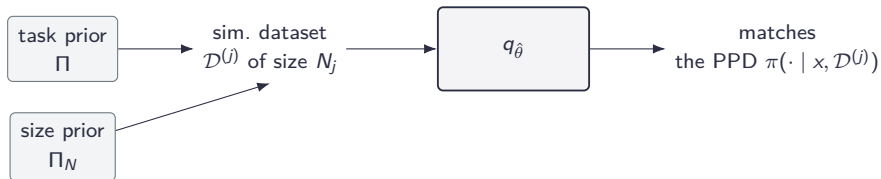
During pre-training we repeatedly: **draw a task, simulate a dataset of random size N_j** , and nudge the net $q_{\hat{\theta}}$ to match that dataset's PPD.



For **TabPFN**: $\Pi_N = \text{uniform on } \{1, \dots, 1023\}$. (why so small? next-to-last slide.)

Setup: a PFN is trained to copy the PPD, at random sizes

During pre-training we repeatedly: **draw a task, simulate a dataset of random size N_j** , and nudge the net $q_{\hat{\theta}}$ to match that dataset's PPD.



For **TabPFN**: $\Pi_N = \text{uniform on } \{1, \dots, 1023\}$. (why so small? next-to-last slide.)

Key question: as the size N changes, does the *ideal* network change too?

Fact 1 — the target's complexity *grows* with n

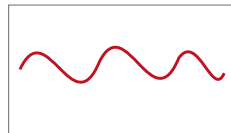
The thing we copy is the PPD $\pi(y \mid x, \mathcal{D}_n)$. How wiggly it is depends on n :



$n = 1$
 $\approx$ prior mean
(constant)



$n = 30$
gentle structure

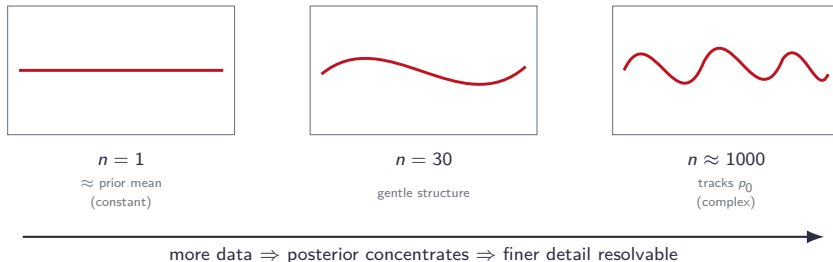


$n \approx 1000$
tracks p_0
(complex)

more data $\Rightarrow$ posterior concentrates $\Rightarrow$ finer detail resolvable

Fact 1 — the target's complexity *grows* with n

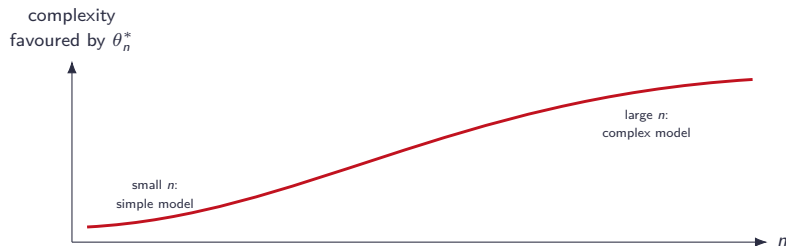
The thing we copy is the PPD $\pi(y \mid x, \mathcal{D}_n)$. How wiggly it is depends on n :



$n = 1$: one point can't reveal structure, so the PPD is essentially the average prior model — nearly flat.

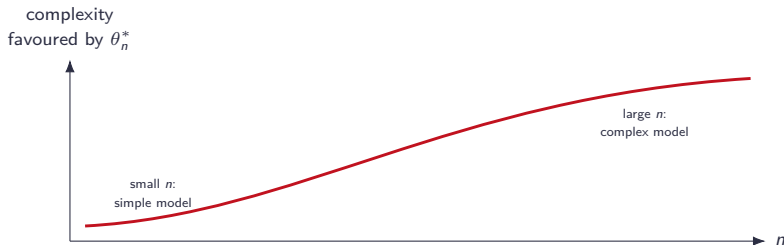
Fact 2 — so the *ideal parameters* follow the size

Define the KL-optimal parameter *for a fixed size* n : $\theta_n^* = \arg \max_{\theta} \mathbb{E}_{\Pi} [\log q_{\theta}(Y | X, \mathcal{D}_n)]$.



Fact 2 — so the *ideal parameters* follow the size

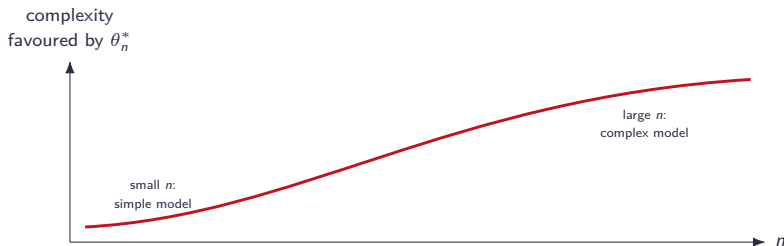
Define the KL-optimal parameter *for a fixed size* n : $\theta_n^* = \arg \max_{\theta} \mathbb{E}_{\Pi} [\log q_{\theta}(Y | X, \mathcal{D}_n)]$.



The target $\pi(\cdot | x, \mathcal{D}_n)$ changes with $n \Rightarrow$ its best approximator θ_n^* does too: **bigger sets justify (and demand) more complexity.**

Fact 2 — so the *ideal parameters* follow the size

Define the KL-optimal parameter *for a fixed size* n : $\theta_n^* = \arg \max_{\theta} \mathbb{E}_{\Pi} [\log q_{\theta}(Y | X, \mathcal{D}_n)]$.



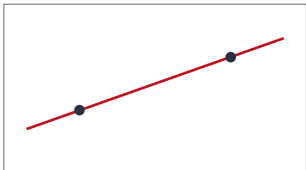
The target $\pi(\cdot | x, \mathcal{D}_n)$ changes with $n \Rightarrow$ its best approximator θ_n^* does too: **bigger sets justify (and demand) more complexity.**

But training optimises an *average* over $N_j \sim \Pi_N$: $\hat{\theta}$ blends the $\theta_{N_j}^*$. **So Π_N sets the average complexity.**

The analogy that makes it click: #points caps the fit

Fit a curve to data. The *number of points* alone tells you how complex a fit is sensible.

2 points

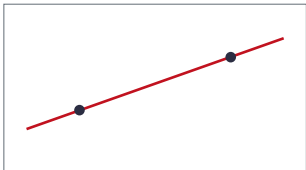


a line is all you
can justify

The analogy that makes it click: #points caps the fit

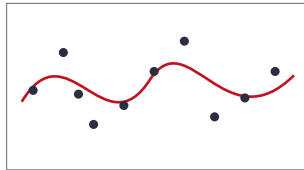
Fit a curve to data. The *number of points* alone tells you how complex a fit is sensible.

2 points



a line is all you
can justify

many points

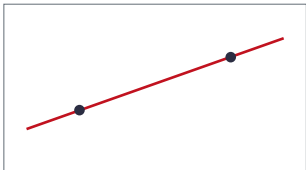


a rich, wiggly curve
is now justified

The analogy that makes it click: #points caps the fit

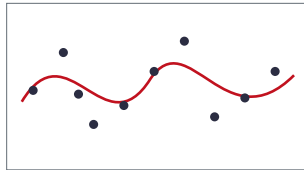
Fit a curve to data. The *number of points* alone tells you how complex a fit is sensible.

2 points



a line is all you
can justify

many points



a rich, wiggly curve
is now justified

A degree-9 polynomial through 2 points is nonsense (over-fitting). The PPD, being Bayes-optimal, **automatically** stays simple when data is scarce.

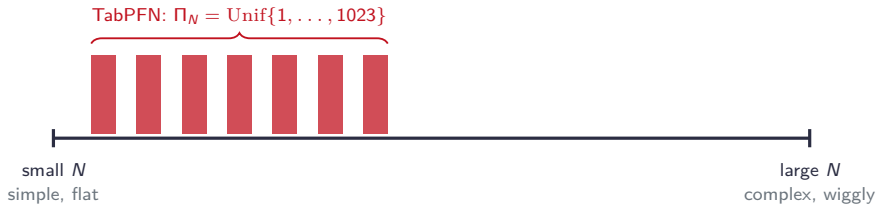
Therefore: Π_N is a complexity dial = a regularizer

Choosing where Π_N puts its mass is choosing how complex the network's predictions become.



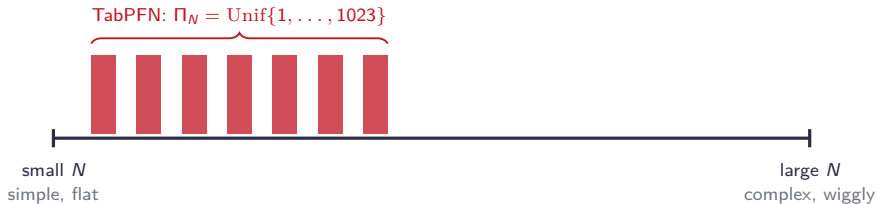
Therefore: Π_N is a complexity dial = a regularizer

Choosing where Π_N puts its mass is choosing how complex the network's predictions become.



Therefore: Π_N is a complexity dial = a regularizer

Choosing where Π_N puts its mass is choosing how complex the network's predictions become.



Same role as a classic regularizer — it trades off complexity:

ridge λ tree depth kernel bandwidth

... and now: **the size prior Π_N**

One subtlety — don't misstate this in Q&A

Ordinary regularizer

- adds a **penalty on the parameters** (e.g. $+\lambda\|\theta\|^2$)
- acts *directly* on the model

One subtlety — don't misstate this in Q&A

Ordinary regularizer

- adds a **penalty on the parameters** (e.g. $+\lambda\|\theta\|^2$)
- acts *directly* on the model

Here (Π_N)

- **no penalty on θ at all**
- you reshape the *training distribution* so the *optimal target* is simpler/more complex
- regularizes **indirectly**

One subtlety — don't misstate this in Q&A

Ordinary regularizer

- adds a **penalty on the parameters** (e.g. $+\lambda\|\theta\|^2$)
- acts *directly* on the model

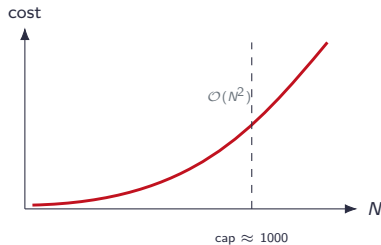
Here (Π_N)

- **no penalty on θ at all**
- you reshape the *training distribution* so the *optimal target* is simpler/more complex
- regularizes **indirectly**

The *effect* on realised complexity is the same — that is why he calls it a regularizer — but the *mechanism* is “control the data sizes,” not “penalize the weights.”

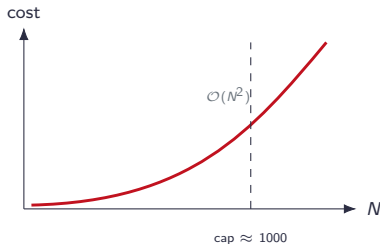
Why it matters — this loads the spring for the puzzle

TabPFN caps Π_N at ≈ 1000 **purely for compute**: a transformer costs $\mathcal{O}(N^2)$.



Why it matters — this loads the spring for the puzzle

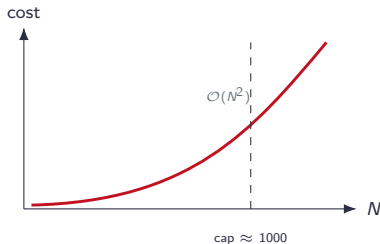
TabPFN caps Π_N at ≈ 1000 **purely for compute**: a transformer costs $\mathcal{O}(N^2)$.



So the net is **regularized to be no more complex than ≤ 1000 -point targets require**.

Why it matters — this loads the spring for the puzzle

TabPFN caps Π_N at ≈ 1000 **purely for compute**: a transformer costs $\mathcal{O}(N^2)$.



So the net is **regularized to be no more complex than ≤ 1000 -point targets require**.

The puzzle, sharpened: a network whose complexity was tuned for ≤ 1000 points *still improves* when fed far more.

⇒ Section 5 (frequentist view) answers *how*: mostly by **shrinking variance**, not by inventing new complexity.

N small $\Rightarrow$ simple target $\Rightarrow$ simple net.

The *size prior* Π_N is the complexity dial.